

Path-Divided-Difference Trend Filtering on Graphs

`gflow` graph trend filtering project notes

Build 2026-05-16 17:02:23 EDT

Abstract

This note proposes a path-divided-difference graph trend-filtering penalty for `gflow`. The goal is to keep the essential behavior of classical one-dimensional trend filtering on path graphs while extending it to metric graphs with branches. The central construction is simple: enumerate local graph paths, turn each path into a one-dimensional coordinate system using edge lengths, compute the ordinary divided difference along that path, and penalize the absolute values of those pathwise differences. The centerpiece is a three-arm star graph, the smallest example where graph trend filtering must decide what “linear” should mean at a branch point.

1 Purpose

The current `gflow` graph trend-filtering implementation follows the recursive graph-operator construction

$$\Delta^{(k+1)} = \begin{cases} L^{(k+1)/2}, & k \text{ odd,} \\ DL^{k/2}, & k \text{ even,} \end{cases}$$

where D is an oriented incidence matrix and $L = D^T D$ is a graph Laplacian. This construction is natural for general graphs, but it does not reproduce classical one-dimensional trend filtering on a path graph for higher orders. In particular, the current graph order-2 operator behaves more like an edge difference of graph Laplacian values than like the third divided difference used by one-dimensional quadratic trend filtering.

This document proposes a second operator family:

$$\Delta_{G,\text{path}}^{(r)},$$

where r is the divided-difference order. The intended relation to trend filtering order k is

$$r = k + 1.$$

Thus $k = 0$ penalizes first differences, $k = 1$ penalizes second divided differences, and $k = 2$ penalizes third divided differences.

2 Classical Divided Differences

Let

$$t_0 < t_1 < \cdots < t_r$$

be positions on the real line, and let $z_j = f(t_j)$. The first divided difference is

$$[t_0, t_1]f = \frac{z_1 - z_0}{t_1 - t_0}.$$

The second divided difference is

$$[t_0, t_1, t_2]f = \frac{[t_1, t_2]f - [t_0, t_1]f}{t_2 - t_0}.$$

Some conventions multiply this expression by 2, so that it equals the ordinary quadratic coefficient scale. The scaling convention matters for λ , but not for the fitted path after the tuning grid is adjusted. The third divided difference is defined recursively:

$$[t_0, t_1, t_2, t_3]f = \frac{[t_1, t_2, t_3]f - [t_0, t_1, t_2]f}{t_3 - t_0}.$$

The general recursive definition is

$$[t_0, \dots, t_r]f = \frac{[t_1, \dots, t_r]f - [t_0, \dots, t_{r-1}]f}{t_r - t_0}.$$

The key fact is the null-space property:

$$[t_0, \dots, t_r]f = 0 \quad \text{for every consecutive window}$$

if the sampled values lie on a polynomial of degree at most $r - 1$. This is the algebra behind one-dimensional trend filtering.

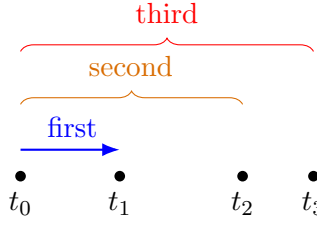


Figure 1: Divided differences are local polynomial detectors. A second divided difference measures curvature; a third divided difference measures change in curvature.

3 Metric Graphs and Path Coordinates

Let

$$G = (V, E, \ell)$$

be an undirected graph with positive edge lengths

$$\ell_{uv} > 0, \quad (u, v) \in E.$$

For a simple graph path

$$p = (v_0, v_1, \dots, v_r),$$

define path-induced coordinates

$$t_0 = 0, \quad t_j = \sum_{m=1}^j \ell_{v_{m-1}v_m}, \quad j = 1, \dots, r.$$

The pathwise divided difference is then

$$\Delta_p^{(r)} f = [t_0, \dots, t_r] (f_{v_0}, \dots, f_{v_r}).$$

This is an ordinary one-dimensional divided difference, but the coordinate axis is induced by graph edge lengths along the path.

4 Canonical Path Rows

For fixed order r , let

$$\vec{\mathcal{P}}_r(G) = \{(v_0, \dots, v_r) : v_i \text{ distinct and } (v_{i-1}, v_i) \in E\}$$

be the set of directed simple length- r paths. Reversing a path does not change the absolute value of the divided difference except for an order-dependent sign. Since the trend-filtering penalty uses absolute values, the two rows

$$(v_0, \dots, v_r) \quad \text{and} \quad (v_r, \dots, v_0)$$

are duplicates for penalty purposes.

Define the reversal equivalence relation

$$(v_0, \dots, v_r) \sim (v_r, \dots, v_0),$$

and let

$$\mathcal{P}_r^{\text{can}}(G) = \vec{\mathcal{P}}_r(G) / \sim$$

be the canonical undirected path-row set. In computation, each equivalence class can be represented by the lexicographically smaller orientation.

This canonicalization has a practical purpose. On a path graph, classical one-dimensional trend filtering uses each consecutive window once. If both orientations were retained, the penalty would be multiplied by two and the same numerical λ grid would no longer match the 1D reference solver.

5 The Path-Divided-Difference Penalty

The proposed penalty has the form

$$\left\| \Delta_{G, \text{path}}^{(r)} f \right\|_1 = \sum_{p \in \mathcal{P}_r^{\text{can}}(G)} w_p \left| \Delta_p^{(r)} f \right|.$$

The weights w_p are not conductances in the low-pass smoothing sense. They are row weights for the ℓ_1 graph trend-filtering penalty.

5.1 Unit Weighting

The simplest choice is

$$w_p = 1.$$

This is the natural default when exact path-graph parity is the priority. On a path graph with vertices ordered along the path, it gives the same rows as the classical divided-difference trend-filtering operator.

5.2 Anchor-Mean Weighting

For graphs with highly unequal branching, one may want to prevent vertices with many outgoing path continuations from dominating the penalty. Let

$$a(p)$$

be the first vertex of the chosen canonical representative of p , and let

$$\mathcal{P}_{r,u}^{\text{can}}(G) = \{p \in \mathcal{P}_r^{\text{can}}(G) : a(p) = u\}.$$

The anchor-mean choice is

$$w_p = \frac{1}{|\mathcal{P}_{r,a(p)}^{\text{can}}(G)|}.$$

The corresponding penalty can be written as

$$\left\| \Delta_{G,\text{path}}^{(r)} f \right\|_{1,\text{anchor}} = \sum_{u \in V} \frac{1}{|\mathcal{P}_{r,u}^{\text{can}}(G)|} \sum_{p \in \mathcal{P}_{r,u}^{\text{can}}(G)} \left| \Delta_p^{(r)} f \right|,$$

with the convention that vertices with no length- r paths contribute zero.

In an R interface, this suggests

```
path.weighting = c("unit", "anchor.mean").
```

6 Exact Agreement on Path Graphs

Consider a path graph

$$1 - 2 - \dots - n$$

with edge lengths

$$h_i = \ell_{i,i+1}.$$

The canonical length- r path rows are exactly

$$(1, 2, \dots, r+1), \quad (2, 3, \dots, r+2), \quad \dots, \quad (n-r, \dots, n).$$

The path-induced coordinates are

$$0, \quad h_i, \quad h_i + h_{i+1}, \quad \dots, \quad \sum_{m=i}^{i+r-1} h_m.$$

Therefore

$$\Delta_{G,\text{path}}^{(r)}$$

with unit weighting is exactly the nonuniform one-dimensional divided-difference operator on the path.

Consequence. If `fit.graph.trend.filtering()` uses this operator on a path graph, then order k graph trend filtering with $r = k + 1$ can be made algebraically identical to the corresponding `genlasso` trend filter, up to the same divided-difference scaling convention and the same λ selection rule.

7 The Star Graph Is the Key Test Case

The smallest graph that is not merely a path is a star. To test an order $k = 1$ trend filter, whose penalty uses $r = 2$ second divided differences, the smallest useful star has two vertices on each arm:

$$u - v_{a1} - v_{a2}, \quad a = 1, 2, 3.$$

Here u is the center. The graph has one shared branch point and three arms.

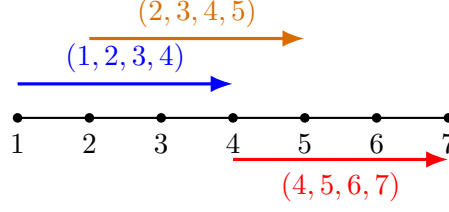


Figure 2: For $r = 3$, canonical path rows on a path graph are exactly the sliding four-point windows of one-dimensional quadratic trend filtering.

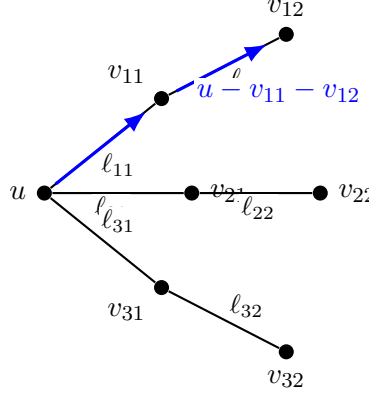


Figure 3: The three-arm star with two vertices per arm. It is the simplest graph where a path-divided-difference penalty must decide whether smoothness should be enforced along arms, across arms, or both.

7.1 Why Cross-Arm Paths Are Not Innocent

If we include every simple length-2 path, then paths such as

$$v_{11} - u - v_{21}$$

are included. With equal edge lengths, zero second difference on all such cross-arm paths imposes equations like

$$f_{11} - 2f_u + f_{21} = 0.$$

Combining all cross-arm equations forces compatibility between slopes on different arms. On a three-arm star with only one vertex per arm, it can collapse the order-1 null space to constants.

That may be desirable for some embedded problems, but it is not the desired first implementation here. If the graph represents a branching metric object, then a branch point should share a function value, while each outgoing arm may have its own slope.

7.2 Arm-Continuation Paths

For the first implementation, the intended second-difference paths at the center are the arm-continuation paths

$$(u, v_{11}, v_{12}), \quad (u, v_{21}, v_{22}), \quad (u, v_{31}, v_{32}).$$

With equal edge lengths, zero second divided difference gives

$$f_u - 2f_{11} + f_{12} = 0,$$

$$\begin{aligned} f_u - 2f_{21} + f_{22} &= 0, \\ f_u - 2f_{31} + f_{32} &= 0. \end{aligned}$$

Equivalently, for each arm a ,

$$f_{a2} - f_{a1} = f_{a1} - f_u.$$

This says that each arm is locally linear as a function of distance from the center.

7.3 The Null Space on the Star

For equal edge lengths, the three equations imply

$$f_{a1} = f_u + s_a, \quad f_{a2} = f_u + 2s_a, \quad a = 1, 2, 3.$$

Thus the null space has four degrees of freedom:

$$f_u, \quad s_1, \quad s_2, \quad s_3.$$

The center value f_u is shared, but the three arms may have three different slopes.

quantity	meaning
f_u	shared intercept at the branch point
s_1	slope along arm 1
s_2	slope along arm 2
s_3	slope along arm 3

This is the behavior we want the operator to have in the first implementation: linear along each arm, coupled only through the common value at the branch point.

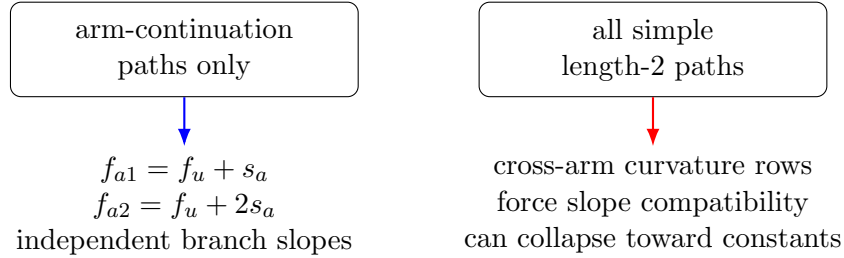


Figure 4: The star graph exposes the modeling choice. Arm-continuation paths preserve branch-specific slopes; all simple paths also penalize turns through the branch point.

7.4 Unequal Edge Lengths

Let the two edge lengths on arm a be

$$h_{a1} = \ell_{u, v_{a1}}, \quad h_{a2} = \ell_{v_{a1}, v_{a2}}.$$

The zero second divided-difference condition along the arm is

$$\frac{\frac{f_{a2} - f_{a1}}{h_{a2}} - \frac{f_{a1} - f_u}{h_{a1}}}{h_{a1} + h_{a2}} = 0.$$

Therefore

$$\frac{f_{a2} - f_{a1}}{h_{a2}} = \frac{f_{a1} - f_u}{h_{a1}}.$$

The slope along arm a is constant in metric distance, but the slopes may still differ across arms.

8 A First Implementation Path Family

The star example shows that path enumeration is not merely an implementation detail. The first implementation should therefore use a deliberate path family rather than blindly using every simple path.

For the initial `gflow` implementation, define

$$\mathcal{P}_r^{\text{branch}}(G) \subseteq \mathcal{P}_r^{\text{can}}(G)$$

to contain canonical paths that continue along a local branch rather than turning through a high-degree vertex. On a path graph,

$$\mathcal{P}_r^{\text{branch}}(G) = \mathcal{P}_r^{\text{can}}(G),$$

so exact 1D parity is preserved. On a star, length-2 rows include arm continuations such as

$$u - v_{a1} - v_{a2},$$

but exclude cross-arm rows such as

$$v_{a1} - u - v_{b1}, \quad a \neq b.$$

This suggests an implementation option:

```
operator.family = "path.divided.difference",
path.family = c("branch.continuation", "all.simple"),
path.weighting = c("unit", "anchor.mean").
```

The recommended first default is

```
path.family = "branch.continuation",    path.weighting = "unit".
```

9 Estimator

Given a selected path family $\mathcal{P}_r^*(G)$, the estimator is

$$\hat{f}_\lambda = \arg \min_{f \in \mathbb{R}^{|V|}} \left\{ \frac{1}{2} \|y - f\|_2^2 + \lambda \sum_{p \in \mathcal{P}_r^*(G)} w_p |\Delta_p^{(r)} f| \right\}.$$

Equivalently, if $A_{r,\text{path}}$ is the sparse matrix whose rows are the weighted divided-difference path rows, then

$$\hat{f}_\lambda = \arg \min_f \left\{ \frac{1}{2} \|y - f\|_2^2 + \lambda \|A_{r,\text{path}} f\|_1 \right\}.$$

This is a generalized lasso problem. It can use the same broad computational framework as the current `fit.graph.trend.filtering()` implementation, provided the penalty matrix can be constructed and passed to the solver.

10 Comparison With Recursive Graph Trend Filtering

The recursive graph operator and the path-divided-difference operator encode different meanings of smoothness.

Recursive graph operator	Path-divided-difference operator
Uses incidence and Laplacian powers.	Uses ordinary divided differences along selected graph paths.
Natural from graph calculus.	Natural from metric path geometry.
On a path, higher orders need not match classical 1D trend filtering.	On a path, unit-weighted canonical rows reproduce classical 1D trend filtering.
Branch behavior is determined by D and L .	Branch behavior is determined by the chosen path family.

The important point is not that one construction is universally better. They are different modeling assumptions. The path-divided-difference construction is designed for experiments where graph edges are interpreted as metric segments and smoothness should be measured along local graph paths.

11 Testing Implications

The proposed operator family has unusually clear tests.

Path parity tests. On path graphs with unit weighting, compare the constructed penalty matrix against the one-dimensional divided-difference matrix used by a reference implementation. This should be tested for equal and unequal edge lengths.

Canonicalization tests. Verify that a path and its reverse produce only one row. This prevents accidental doubling of the penalty and keeps the λ scale comparable to one-dimensional trend filtering.

Star null-space tests. For the three-arm star with two vertices per arm, the $r = 2$ branch-continuation operator should annihilate vectors satisfying

$$f_{a1} = f_u + s_a, \quad f_{a2} = f_u + 2s_a.$$

It should not force $s_1 = s_2 = s_3$.

Cross-arm contrast tests. If `path.family = "all.simple"` is implemented, it should include cross-arm rows and therefore have a different null space on the star graph. This contrast is a useful guard against silently changing the branch-continuity semantics.

12 Open Modeling Questions

Several choices should remain explicit until empirical benchmarks and small graph algebra agree.

1. Should `path.weighting = "unit"` remain the default because it gives the cleanest 1D parity?

2. Should `path.family = "branch.continuation"` be the default for metric graph data?
3. How should branch-continuation paths be detected in graphs with cycles, noisy neighborhoods, or no obvious arm structure?
4. Should cross-arm paths be available as an explicit stronger-smoothness option?
5. For higher orders, should the first implementation limit r to 1, 2, 3, where path enumeration and diagnostics remain transparent?

13 Summary

The path-divided-difference graph trend-filtering penalty is built from local one-dimensional divided differences along selected graph paths:

$$\left\| \Delta_{G, \text{path}}^{(r)} f \right\|_1 = \sum_{p \in \mathcal{P}_r^*(G)} w_p |\Delta_p^{(r)} f|.$$

With canonical path rows and unit weighting, it reproduces ordinary one-dimensional trend filtering on path graphs. On branched graphs, the selected path family becomes the modeling decision. The three-arm star shows why this matters: branch-continuation paths allow one shared center value and independent arm slopes, while all simple paths impose stronger cross-arm compatibility. This makes the star graph the natural first analytic and unit test for the proposed `gflow` operator family.